Smart office preferences manager

PRIOT Project

Author:

Dragomir Andrei-Mihai 343C1

Description:

Documentation for the design and implementation of a smart in office logger, an IoT system for managing employee access and monitoring environmental conditions. The system integrates hardware components such as RFID, sensors, and an ESP32 microcontroller, and communicates with a server for centralized data visualization and management.

January 13, 2025

Contents

Chapter 1

Introduction

This paper describes the implementation of an IoT system for managing employee access and monitoring the environment in the office (temperature and humidity). The project combines a sensor network with an ESP32 hub that communicates with a central server to store and visualize data.

Employees receive alerts when the temperature or humidity is not optimal for their inputted preferences. The system also logs access events, allowing administrators to monitor employee activity: logins, logouts, access attempts, time spent in the office, etc.

1.1 Key Objectives

- Manage employee access through an RFID system.
- Monitor the environment using a DHT11 sensor.
- Visualize collected data via an interactive web interface.

Chapter 2

Architecture

2.1 Overview

This project architecture is designed to achieve modularity and scalability by separating the ESP32 microcontroller logic from the backend server and frontend interface. It uses a combination of hardware, communication protocols, and software layers to deliver the functionality.

2.1.1 Backend (Flask Server)

The backend is implemented using Python and Flask. It serves as the central hub for processing data received from the ESP32, managing user preferences, and rendering the web-based interface for administrators and employees. Key components include:

- **server.py**: The Flask application that handles HTTP requests and serves dynamic HTML pages.
- **templates/**: Contains HTML files used by Flask for dynamic content rendering (e.g., `preferences.html`, `records.html`).
- **static/**: Stores frontend assets such as:
  - **CSS**: Styling for the interface (e.g., `style.css`).
  - **JavaScript**: Frontend logic and interactivity (e.g., `script.js`).
- **users.json**: A lightweight storage file for user data and preferences.

2.1.2 ESP32 Firmware (C++)

The ESP32 firmware is written in written in C++. This firmware handles the hardware peripherals and communicates with the Flask server and is organized into several files within the `src/` directory:

- **main.cpp**: The entry point for the ESP32 firmware, initializing all components and managing the main logic loop.
- **access_utils.cpp**: Handles RFID-based access control and logs employee activities such as logins and logouts.
- **http_utils.cpp**: Manages HTTP communication between the ESP32 and the Flask server for data exchange.
- **led_buzz_utils.cpp**: Controls visual and auditory feedback through RGB LEDs and a buzzer.
- **temp_utils.cpp**: Captures and processes temperature and humidity data using the DHT11 sensor.
- **rtc_utils.cpp**: Provides accurate timestamps using the RTC module.
- **print_utils.cpp** and **utils.hpp**: Utility functions for logging, debugging, and reusability.

2.2 Hardware and Network Schema

The system uses an ESP32 microcontroller to interact with peripherals and communicate with the backend server over Wi-Fi. The topology is represented as follows:

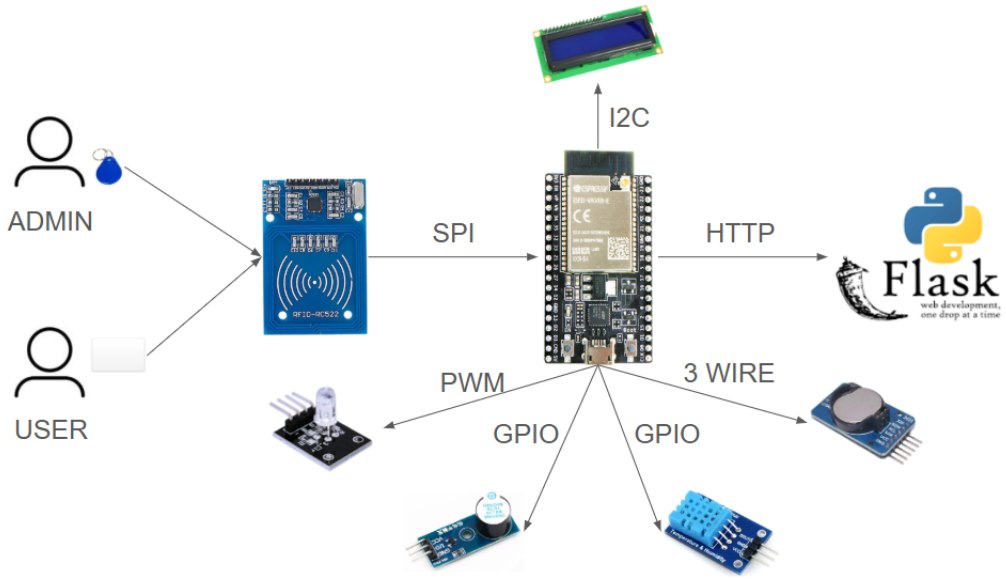


Figure 2.1: Hardware Schema

2.2.1 Hardware and Protocols

The system is designed to integrate various hardware components with the ESP32 microcontroller, which serves as the central hub for communication and control. The following describes the key components and their connections:

- **RFID Reader (SPI Protocol):** The RFID module communicates with the ESP32 using the high-speed SPI protocol. It is used for user authentication and access control. Admins and users interact with the system by scanning their RFID tags or cards.
- **LCD Screen (I2C Protocol):** The LCD display is connected to the ESP32 via the I2C protocol. It provides a user interface for displaying system status, access logs, and environmental data locally.
- **Flask Server (HTTP over Wi-Fi):** The ESP32 communicates with the Flask server using the HTTP protocol over Wi-Fi. This connection

is used to send collected data (e.g., RFID access logs and environmental readings) and receive configurations or updates from the server.

- **RGB LED (PWM):** The RGB LED is connected to the ESP32 through GPIO pins configured for PWM (Pulse Width Modulation). It provides visual feedback for operations such as successful logins, errors, or status updates.
- **Buzzer (GPIO):** The buzzer is directly connected to a GPIO pin on the ESP32. It provides auditory feedback for system events, such as login success, login failure, or alerts.
- **DHT11 Sensor (GPIO):** The DHT11 sensor is linked to the ESP32 through a GPIO pin. It captures environmental data, specifically temperature and humidity, which is logged during user interactions.
- **RTC Module (Three-Wire Protocol):** The Real-Time Clock (RTC) module connects to the ESP32 using a three-wire protocol, providing accurate timestamps for logging access events and environmental data.

This hardware configuration ensures good communication and reliable data exchange between the various components, with the ESP32 acting as the central coordinator.

Chapter 3

Implementation

This system is implemented using a combination of C++ for the ESP32 firmware and Python for the Flask server. The following sections detail the key components and their functionalities.

3.1 ESP32 Firmware

3.2 Introduction

The ESP32 firmware is designed to manage user access, handle environmental data, and provide feedback through LEDs and an LCD display. It also implements secure communication with a Flask server using encrypted requests and responses. This document describes the implementation structure and details for the firmware.

3.3 Project Structure

The firmware is organized into the following files:

- **main.cpp**: Entry point of the firmware.
- **access_utils.cpp**: Handles user access-related functionalities.
- **http_utils.cpp**: Manages HTTP requests and server interactions.
- **encryption.cpp**: Implements AES encryption and decryption.
- **led_buzz_utils.cpp**: Controls LED and buzzer behaviors.

- **print_utils.cpp**: Handles LCD printing.
- **rtc_utils.cpp**: Real-time clock management.
- **temp_utils.cpp**: Reads and processes temperature and humidity data.
- **utils.cpp, utils.hpp**: Core utilities and global declarations.

3.4 Hardware Configuration

3.4.1 Pin Assignments

- **LEDs**: RED_PIN (17), GREEN_PIN (4), BLUE_PIN (2)
- **Buzzer**: BUZZER_PIN (5)
- **Joystick**: SW_PIN (34), URX_PIN (32), URY_PIN (35)
- **RTC**: CLK_PIN (33), DAT_PIN (16), RST_PIN (15)
- **LCD**: SDA_PIN (25), SCL_PIN (26)
- **DHT Sensor**: DHTPIN (13)

3.4.2 Peripherals

The following peripherals are utilized:

- RFID Module (MFRC522)
- LCD Display (LiquidCrystal_I2C)
- Real-Time Clock (DS1302)
- DHT11 Temperature and Humidity Sensor

3.5 Core Functionalities

3.5.1 User Access Management

- **Add Access**: Allows new users to gain access by storing their UID.
- **Remove Access**: Revokes access for specific users.
- **Authorization**: Checks if a UID is authorized for access.

- Functions:
  - `void addCardAccess()`
  - `void removeCardAccess()`
  - `bool isAuthorizedUID(String uid)`

3.5.2 Temperature and Humidity Monitoring

- Reads data from the DHT11 sensor.
- Provides real-time feedback on the LCD display.
- Functions:
  - `void get_temperature_humidity(float &temperature, float &humidity)`
  - `void print_temperature_humidity(float temperature, float humidity)`

3.5.3 Real-Time Clock Management

- Synchronizes and reads date/time.
- Converts date/time to integer format for calculations.
- Functions:
  - `RtcDateTime readQuartzTime()`
  - `void setDateTime()`

3.5.4 LED and Buzzer Feedback

- Provides visual and audio feedback based on events.
- Functions:
  - `void turnOffLEDs()`
  - `void goodbyeMelody()`
  - `void accessGrantedMelody()`
  - `void accessDeniedMelody()`

3.5.5 HTTP and HTTPS Communication

- Communicates with the Flask server using HTTPS.
- Functions:
  - `void sendEmployeeData(String name, String uid, String time, float temperature, float humidity, String reminder, bool inOut)`
  - `void fetchUserPreferences(String uid, float &temperature, float &humidity)`

3.6 Alerts and Notification System

The alerts in this implementation are designed to ensure that users and administrators are promptly informed about important events and status updates within the platform.

3.6.1 System Overview

The alerts and notification system monitors key events, such as unauthorized access attempts and changes in user preferences. Upon detecting such events, the system generates alerts that are sent to the ESP32 board.

3.6.2 Implementation on ESP32

The ESP32 firmware includes functions to trigger audible and visual alerts using LEDs and buzzers. For example:

- **LED Notifications**: RGB LEDs are programmed to display specific colors based on event types (e.g., red for errors, green for successful operations, and blue for general notifications).
- **Audible Alerts**: The buzzer emits predefined melodies or tones to indicate critical alerts, such as unauthorized card access or temperature anomalies.

Additionally, the ESP32 sends encrypted notification payloads to the Flask server for centralized logging and processing.

3.7 LCD Display Management

- Displays messages and feedback on the LCD.
- Functions:
  - `void printStringOnLCD(const char *message)`
  - `void showSettingPreferences(float currentTemp, float currentHumid, float preferedTemp, float preferedHumid)`

3.8 Main Entry Point

- Initializes peripherals, connects to WiFi, and starts the HTTP server.
- Functions:
  - `void connectToWiFi()`
  - `void setupLEDControl()`

Chapter 4

Visualization and Data Processing

The Flask server provides visualization and data processing functionalities, offering insights into employee activities, preferences, and environmental conditions. The implementation is structured to process and visualize data dynamically based on the logs and user data.

4.1 Data Sources

The server processes data from two primary sources:

- **User Data**: Loaded from `users.json`, containing user preferences and access information.
- **Logs**: Stored in `mock_logs.json`, tracking employee login/logout activities along with timestamps and environmental conditions.

4.2 Visualization Dashboard

The visualization dashboard displays various graphs and statistics, accessible through the `/visualization` route. Key components include:

- **User Preferences Chart**: Bar graph showing temperature and humidity preferences for each user.
- **Time Series Data**: Line graphs of temperature and humidity over time.

- **All Hours Graph**: Line graph showing the frequency of entries across different hours of the day.
- **Top Visitors**: Bar graph of the most frequent visitors based on logs.

The data is processed using Python's `Counter` and rendered dynamically using Chart.js.

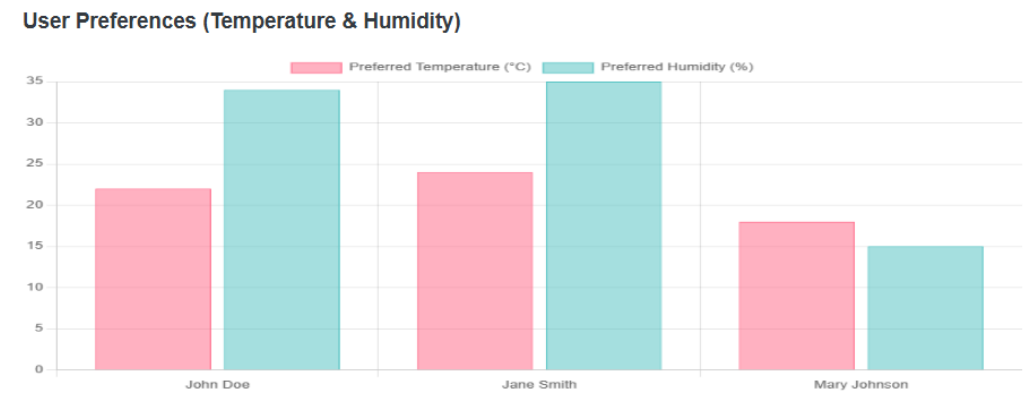


Figure 4.1: User Preferences Temperature & Humidity

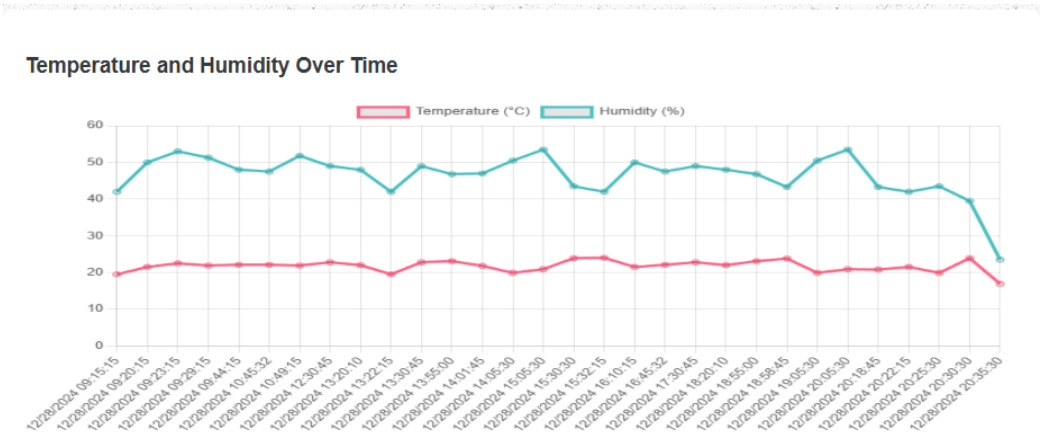


Figure 4.2: Temperature and Humidity Over Time

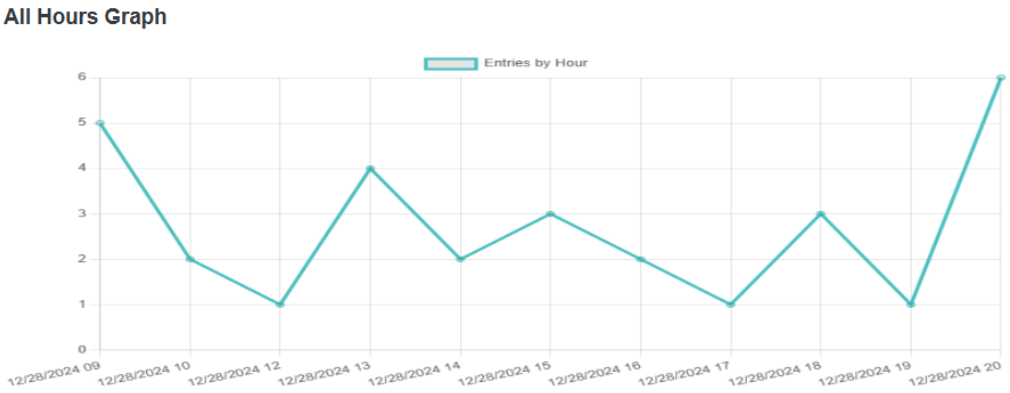


Figure 4.3: All Hours Graph

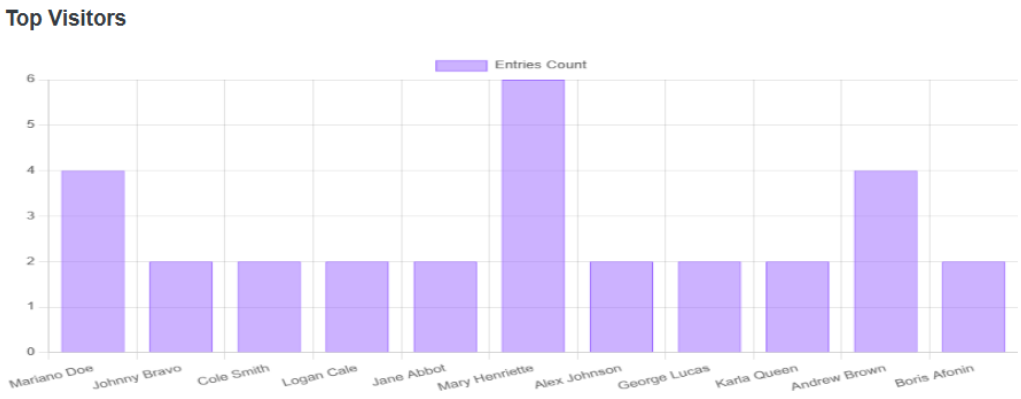
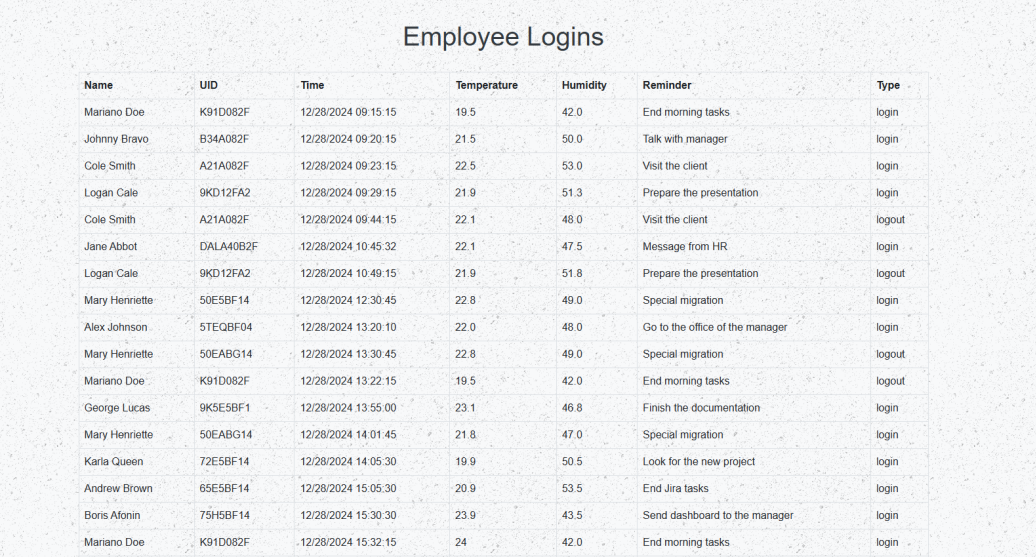


Figure 4.4: Top Visitors

4.3 Records Display

The `/records` route lists all employee activities in a tabular format:

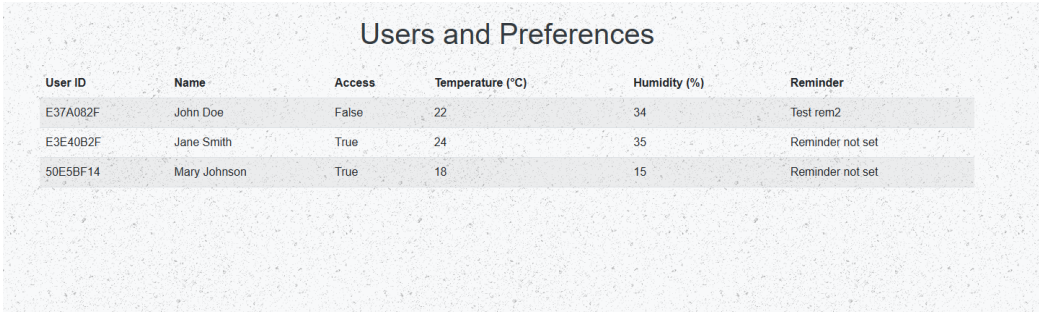
- Name, UID, timestamp, temperature, humidity, reminder, and activity type (login/logout) are shown.



Employee Logins

Name	UID	Time	Temperature	Humidity	Reminder	Type
Mariano Doe	K91D082F	12/28/2024 09:15:15	19.5	42.0	End morning tasks	login
Johnny Bravo	B34A082F	12/28/2024 09:20:15	21.5	50.0	Talk with manager	login
Cole Smith	A21A082F	12/28/2024 09:23:15	22.5	53.0	Visit the client	login
Logan Cale	9KD12FA2	12/28/2024 09:29:15	21.9	51.3	Prepare the presentation	login
Cole Smith	A21A082F	12/28/2024 09:44:15	22.1	48.0	Visit the client	logout
Jane Abbot	DALA40B2F	12/28/2024 10:45:32	22.1	47.5	Message from HR	login
Logan Cale	9KD12FA2	12/28/2024 10:49:15	21.9	51.8	Prepare the presentation	logout
Mary Henriette	50E5BF14	12/28/2024 12:30:45	22.8	49.0	Special migration	login
Alex Johnson	5TEQBF04	12/28/2024 13:20:10	22.0	48.0	Go to the office of the manager	login
Mary Henriette	50EABG14	12/28/2024 13:30:45	22.8	49.0	Special migration	logout
Mariano Doe	K91D082F	12/28/2024 13:22:15	19.5	42.0	End morning tasks	logout
George Lucas	9K5E5BF1	12/28/2024 13:55:00	23.1	46.8	Finish the documentation	login
Mary Henriette	50EABG14	12/28/2024 14:01:45	21.8	47.0	Special migration	login
Karla Queen	72E5BF14	12/28/2024 14:05:30	19.9	50.5	Look for the new project	login
Andrew Brown	65E5BF14	12/28/2024 15:05:30	20.9	53.5	End Jira tasks	login
Boris Afonin	75H5BF14	12/28/2024 15:30:30	23.9	43.5	Send dashboard to the manager	login
Mariano Doe	K91D082F	12/28/2024 15:32:15	24	42.0	End morning tasks	login

Figure 4.5: Records Display



Users and Preferences

User ID	Name	Access	Temperature (°C)	Humidity (%)	Reminder
E37A082F	John Doe	False	22	34	Test rem2
E3E40B2F	Jane Smith	True	24	35	Reminder not set
50E5BF14	Mary Johnson	True	18	15	Reminder not set

Figure 4.6: Registered Users Display

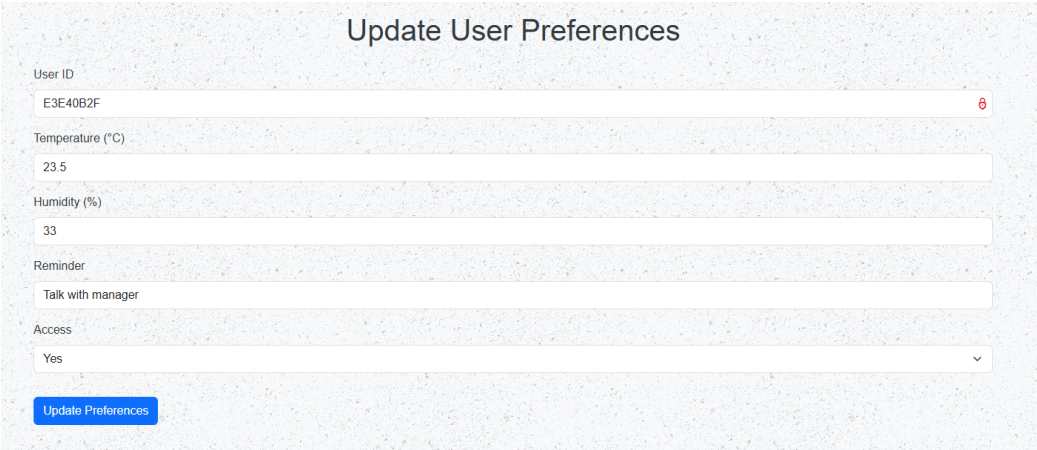


Figure 4.7: Preferences and Access Update Functionality

4.4 Data Processing and Analysis

Time-Based Sorting

Employee logs are sorted based on their timestamps for chronological visualization:

```
sorted_logs = sorted(logs, key=lambda log:
    datetime.strptime(log['time'], "%m/%d/%Y %H:%M:%S"))
```

4.4.1 Statistical Aggregation

Various aggregations are performed using Python's `Counter`:

- Crowded times are derived from timestamps split by hours.
- Top visitors are determined based on the frequency of names in logs.

4.5 Integration with Frontend

The visualization and records pages are rendered dynamically:

- **HTML Templates**: Templates include Bootstrap for styling and Chart.js for visualizations.
- **Dynamic Rendering**: Data from Flask backend is injected into templates using Jinja2, allowing real-time updates.

4.6 User Preferences

The Flask server provides functionalities to manage and update user preferences through the `/preferences` route. This section allows users to:

- View and update individual preferences such as temperature, humidity, and reminders.
- Enable or disable access permissions for specific users.
- Dynamically fetch and modify preferences via the `update_preferences` API.

The preferences are stored in `users.json` and encrypted for secure communication with ESP32 devices. Updates are reflected on the ESP32 firmware in real time.

4.7 Alerts

The system implements an alert mechanism to notify users and employees of specific conditions:

- Alerts consist on update for access or preferences for users which directly show up on ESP32 with a led and buzzer notification, as well as LCD display information.
- Alerts are communicated securely using encrypted messages to ensure data integrity and confidentiality.

Chapter 5

Security

5.1 Access Control

The system implements robust access control mechanisms to manage user permissions effectively:

- Each user is identified by a unique UID, stored securely in the `users.json` file.
- Access permissions are managed through APIs such as `/set_access`, `/add_access`, and `/remove_access`.
- The ESP32 firmware validates UIDs locally to ensure only authorized users can interact with the system.
- Administrative privileges are granted based on predefined UIDs, such as `ADMIN_UID`, to restrict critical actions like resetting the system or modifying global settings.

5.2 Protocols

The system uses secure communication protocols to ensure data integrity and confidentiality:

- HTTPS is used for all communications between the ESP32 and the Flask server to protect against man-in-the-middle attacks.
- The Flask server utilizes a self-signed SSL certificate, ensuring encrypted data exchange.

- The ESP32 employs the WiFiClientSecure library for secure HTTPS requests, supporting modern encryption standards.
- The MQTT protocol can be integrated for real-time notifications, employing TLS for secure messaging.

5.3 Data Encryption

Data security is enforced through AES-128 encryption:

- All sensitive data exchanged between the ESP32 and the Flask server is encrypted using the AES-128 CBC mode.
- A pre-shared key (`aes_key`) and an initialization vector (`iv`) are used for encryption and decryption.
- The system employs PKCS7 padding to ensure plaintext aligns with the block size required by AES.
- Encrypted data is Base64-encoded for compatibility with JSON and HTTP payloads.
- Decryption is performed on the receiving end to reconstruct the original data, ensuring end-to-end security.

Chapter 6

Challenges and Solutions

6.1 Challenges

The development and deployment of the ESP32-based IoT system presented several challenges:

6.1.1 Secure Communication

Implementing secure communication between the ESP32 and the Flask server required careful management of encryption keys and SSL certificates. Debugging encrypted data transmission added complexity due to potential mismatches in encoding and padding.

6.1.2 Resource Constraints

The ESP32's limited memory and processing power posed challenges in handling encryption, parsing large JSON payloads, and managing multiple peripherals concurrently.

6.1.3 Data Synchronization

Ensuring real-time synchronization of user preferences and logs between the server and the ESP32 was challenging, especially during network interruptions or delays.

6.1.4 Error Handling

Properly handling edge cases, such as corrupted payloads, missing data fields, or invalid Base64 encoding, required robust exception handling mechanisms.

6.2 Solutions

6.2.1 Secure Communication

AES-128 encryption was used to protect sensitive data during transmission. HTTPS was employed for secure communication, and extensive debugging ensured compatibility between the ESP32 and Flask server encryption implementations.

6.2.2 Data Synchronization

Retry mechanisms and encrypted acknowledgments were implemented to ensure data integrity during transmission. Logs and user data were periodically synced between the ESP32 and the Flask server.

6.2.3 Comprehensive Debugging and Logging

Debugging tools were used extensively to capture and analyze raw and decrypted data at each stage. Errors were logged both on the ESP32 and the Flask server for easier troubleshooting.

—

Chapter 7

References

- Official ESP32 Documentation: https://www.espressif.com/en/products/socs/esp32/documentation
- Flask Documentation: https://flask.palletsprojects.com/
- Cryptography: AES in Python: https://pycryptodome.readthedocs.io/
- Arduino Libraries: https://www.arduino.cc/reference/en/libraries/
- Chart.js for Visualization: https://www.chartjs.org/
- JSON and REST API Best Practices: https://restfulapi.net/
- Secure Communication with HTTPS: https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview
- Base64 Encoding/Decoding: https://www.base64decode.org/